

# **EC200U&EG91xU Series**

## **QuecOpen(SDK)**

### **(U)SIM Development Guide**

**LTE Standard Module Series**

Version: 1.0

Date: 2023-11-23

Status: Released



At Quectel, our aim is to provide timely and comprehensive services to our customers. If you require any assistance, please contact our headquarters:

**Quectel Wireless Solutions Co., Ltd.**

Building 5, Shanghai Business Park Phase III (Area B), No.1016 Tianlin Road, Minhang District, Shanghai 200233, China

Tel: +86 21 5108 6236

Email: [info@quectel.com](mailto:info@quectel.com)

**Or our local offices. For more information, please visit:**

<http://www.quectel.com/support/sales.htm>.

**For technical support, or to report documentation errors, please visit:**

<http://www.quectel.com/support/technical.htm>.

Or email us at: [support@quectel.com](mailto:support@quectel.com).

## Legal Notices

We offer information as a service to you. The provided information is based on your requirements and we make every effort to ensure its quality. You agree that you are responsible for using independent analysis and evaluation in designing intended products, and we provide reference designs for illustrative purposes only. Before using any hardware, software or service guided by this document, please read this notice carefully. Even though we employ commercially reasonable efforts to provide the best possible experience, you hereby acknowledge and agree that this document and related services hereunder are provided to you on an “as available” basis. We may revise or restate this document from time to time at our sole discretion without any prior notice to you.

## Use and Disclosure Restrictions

### License Agreements

Documents and information provided by us shall be kept confidential, unless specific permission is granted. They shall not be accessed or used for any purpose except as expressly provided herein.

### Copyright

Our and third-party products hereunder may contain copyrighted material. Such copyrighted material shall not be copied, reproduced, distributed, merged, published, translated, or modified without prior written consent. We and the third party have exclusive rights over copyrighted material. No license shall be granted or conveyed under any patents, copyrights, trademarks, or service mark rights. To avoid ambiguities, purchasing in any form cannot be deemed as granting a license other than the normal non-exclusive, royalty-free license to use the material. We reserve the right to take legal action for noncompliance with abovementioned requirements, unauthorized use, or other illegal or malicious use of the material.

## Trademarks

Except as otherwise set forth herein, nothing in this document shall be construed as conferring any rights to use any trademark, trade name or name, abbreviation, or counterfeit product thereof owned by Quectel or any third party in advertising, publicity, or other aspects.

## Third-Party Rights

This document may refer to hardware, software and/or documentation owned by one or more third parties ("third-party materials"). Use of such third-party materials shall be governed by all restrictions and obligations applicable thereto.

We make no warranty or representation, either express or implied, regarding the third-party materials, including but not limited to any implied or statutory, warranties of merchantability or fitness for a particular purpose, quiet enjoyment, system integration, information accuracy, and non-infringement of any third-party intellectual property rights with regard to the licensed technology or use thereof. Nothing herein constitutes a representation or warranty by us to either develop, enhance, modify, distribute, market, sell, offer for sale, or otherwise maintain production of any our products or any other hardware, software, device, tool, information, or product. We moreover disclaim any and all warranties arising from the course of dealing or usage of trade.

## Privacy Policy

To implement module functionality, certain device data are uploaded to Quectel's or third-party's servers, including carriers, chipset suppliers or customer-designated servers. Quectel, strictly abiding by the relevant laws and regulations, shall retain, use, disclose or otherwise process relevant data for the purpose of performing the service only or as permitted by applicable laws. Before data interaction with third parties, please be informed of their privacy and data security policy.

## Disclaimer

- a) We acknowledge no liability for any injury or damage arising from the reliance upon the information.
- b) We shall bear no liability resulting from any inaccuracies or omissions, or from the use of the information contained herein.
- c) While we have made every effort to ensure that the functions and features under development are free from errors, it is possible that they could contain errors, inaccuracies, and omissions. Unless otherwise provided by valid agreement, we make no warranties of any kind, either implied or express, and exclude all liability for any loss or damage suffered in connection with the use of features and functions under development, to the maximum extent permitted by law, regardless of whether such loss or damage may have been foreseeable.
- d) We are not responsible for the accessibility, safety, accuracy, availability, legality, or completeness of information, advertising, commercial offers, products, services, and materials on third-party websites and third-party resources.

**Copyright © Quectel Wireless Solutions Co., Ltd. 2023. All rights reserved.**

# About the Document

## Revision History

| Version | Date       | Author               | Description              |
|---------|------------|----------------------|--------------------------|
| -       | 2021-12-23 | Braden HE/<br>Joe TU | Creation of the document |
| 1.0     | 2023-11-23 | Braden HE/<br>Joe TU | First official release   |

# Contents

|                                                  |           |
|--------------------------------------------------|-----------|
| About the Document.....                          | 3         |
| Contents.....                                    | 4         |
| Table Index.....                                 | 6         |
| Figure Index.....                                | 7         |
| <b>1 Introduction .....</b>                      | <b>8</b>  |
| 1.1. Applicable Modules.....                     | 8         |
| <b>2 API Calling Process .....</b>               | <b>9</b>  |
| <b>3 Differences Between DSDS and DSSS .....</b> | <b>10</b> |
| 3.1. SDK Compilation Command.....                | 10        |
| 3.1. API Usage .....                             | 10        |
| <b>4 (U)SIM API.....</b>                         | <b>12</b> |
| 4.1. Header File.....                            | 12        |
| 4.2. Example File .....                          | 12        |
| 4.3. API Overview .....                          | 12        |
| 4.4. Description of (U)SIM Card API .....        | 13        |
| 4.4.1. ql_sim_get_imsi .....                     | 13        |
| 4.4.1.1. ql_sim_errcode_e.....                   | 14        |
| 4.4.2. ql_sim_get_iccid.....                     | 15        |
| 4.4.3. ql_sim_get_phonenumber .....              | 16        |
| 4.4.4. ql_sim_get_card_status .....              | 17        |
| 4.4.4.1. ql_sim_status_e .....                   | 17        |
| 4.4.5. ql_sim_enable_pin .....                   | 19        |
| 4.4.5.1. ql_sim_verify_pin_info_s.....           | 20        |
| 4.4.6. ql_sim_disable_pin.....                   | 20        |
| 4.4.7. ql_sim_verify_pin .....                   | 20        |
| 4.4.8. ql_sim_change_pin .....                   | 21        |
| 4.4.8.1. ql_sim_change_pin_info_s .....          | 21        |
| 4.4.9. ql_sim_unblock_pin.....                   | 22        |
| 4.4.9.1. ql_sim_unblock_pin_info_s.....          | 22        |
| 4.4.10. ql_sim_set_operate_id.....               | 23        |
| 4.4.11. ql_sim_get_operate_id.....               | 23        |
| 4.4.12. ql_sim_set_hotplug .....                 | 24        |
| 4.4.12.1. ql_sim_hotplug_e .....                 | 24        |
| 4.4.12.2. ql_sim_hotplug_gpio_s .....            | 25        |
| 4.4.13. ql_sim_hotplug_register_cb .....         | 25        |
| 4.4.13.1. ql_sim_hotplug_cb_t .....              | 25        |
| 4.4.13.2. ql_sim_hotplug_status_e .....          | 26        |
| 4.4.14. ql_sim_read_pbk_item.....                | 26        |
| 4.4.14.1. ql_sim_pbk_storage_e .....             | 27        |

---

|                                          |           |
|------------------------------------------|-----------|
| 4.4.14.2. ql_sim_pbk_itemset_info_s..... | 28        |
| 4.4.15. ql_sim_write_pbk_item.....       | 28        |
| 4.4.15.1. ql_sim_pbk_item_info_s.....    | 29        |
| 4.4.16. ql_sim_set_pbk_encoding .....    | 30        |
| 4.4.16.1. ql_sim_pbk_encoding_e .....    | 30        |
| 4.4.17. ql_sim_get_pbk_encoding .....    | 31        |
| 4.4.18. ql_pbk_callback_register .....   | 31        |
| 4.4.18.1. ql_pbk_event_handler_t.....    | 31        |
| <b>5 Appendix References .....</b>       | <b>33</b> |

## Table Index

|                                                                  |    |
|------------------------------------------------------------------|----|
| Table 1: Applicable Modules and (U)SIM Card Support Status ..... | 8  |
| Table 2: API Overview .....                                      | 12 |
| Table 3: Related Documents .....                                 | 33 |
| Table 4: Terms and Abbreviations .....                           | 33 |

## Figure Index

|                                                             |    |
|-------------------------------------------------------------|----|
| Figure 1: (U)SIM API Calling Process.....                   | 9  |
| Figure 2: API Usage Differences Between DSDS and DSSS ..... | 11 |



# 1 Introduction

Quectel LTE Standard EC200U series and EG91xU family modules support QuecOpen® solution. QuecOpen® is an embedded development platform based on RTOS system. It is intended to simplify the design and development of IoT applications. For more information on QuecOpen®, see **document [1]**.

This document is applicable to QuecOpen® solution based on SDK build environment. It provides the application note on the (U)SIM card API, including API calling process, API details, and differences in using DSDS and DSSS.

## 1.1. Applicable Modules

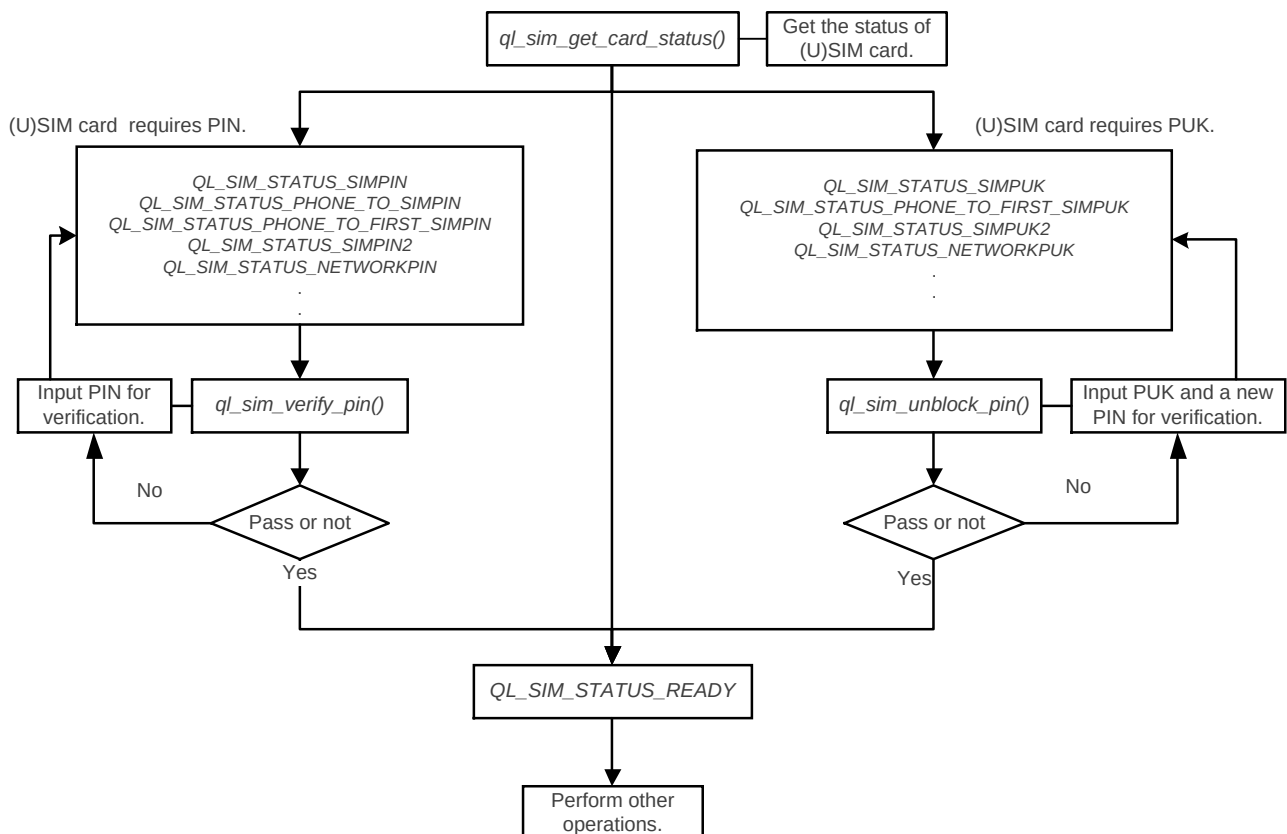
**Table 1: Applicable Modules and (U)SIM Card Support Status**

| Module Family | Module        | (U)SIM Card Support Status                                                 |
|---------------|---------------|----------------------------------------------------------------------------|
| -             | EC200U Series | Default: Single (U)SIM card<br>Through pin multiplexing: Dual (U)SIM cards |
| EG91xU        | EG912U-GL     | Dual (U)SIM cards                                                          |
|               | EG915U Series | Dual (U)SIM cards                                                          |

### NOTE

The modules supporting dual (U)SIM cards can be configured to enable either DSSS or DSDS. See **Chapter 3** for details.

## 2 API Calling Process



**Figure 1: (U)SIM API Calling Process**

The above figure shows the process for getting (U)SIM card status and verifying the PIN. When the (U)SIM card requests a PIN/PIN2/PUK/PUK2 code, you must input the correct PIN/PIN2/PUK/PUK2 code for verification. After the verification, the (U)SIM card will transition to the ready status to perform subsequent operations.

# 3 Differences Between DSDS and DSSS

There are differences on SDK compilation command and API usage between DSDS and DSSS. The details are as follows.

## 3.1. SDK Compilation Command

In SDK compilation command:

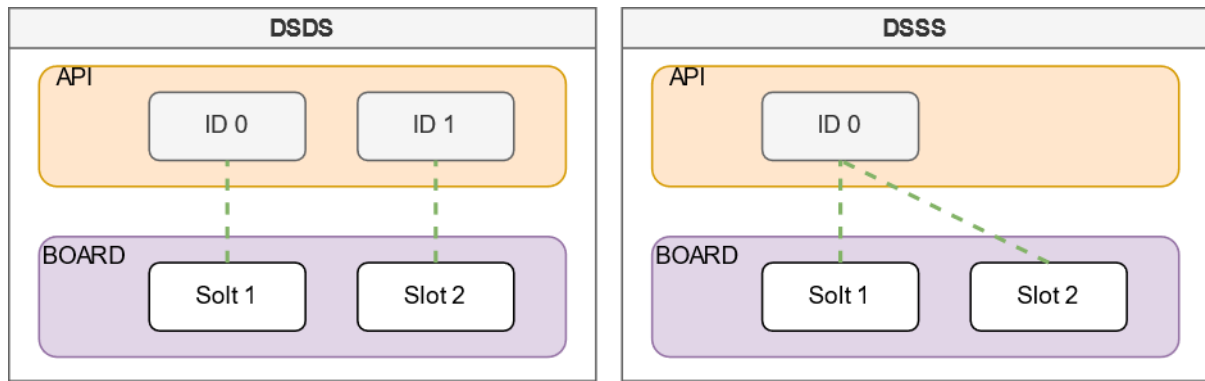
- If the optional parameter *[DSIM]* is set to *DOUBLESIM*, DSDS is enabled.
- If the optional parameter *[DSIM]* is set to *SINGLESIM*, single (U)SIM feature is enabled. If you want to enable DSSS function, DSSS macro control `CONFIG_QUEEC_PROJECT_FEATURE_DSSS` in the kernel should also be enabled; the kernel function is configured through the *target.config* configuration file.

See **document [1]** for details.

## 3.1. API Usage

In DSDS mode, the application layer maps each (U)SIM card slot to a specific (U)SIM ID. You can access a particular (U)SIM card slot by using the corresponding (U)SIM ID.

However in DSSS mode, in addition to the applications related to DSSS switching and hot-plugging, other applications can only detect one (U)SIM card. The behaviour of a DSDS application in use is the same as that of a standard single (U)SIM card application, and the (U)SIM ID remains 0.



**Figure 2: API Usage Differences Between DSDS and DSSS**

For example, let's consider obtaining IMSI of (U)SIM card:

- In DSDS mode, you can obtain the IMSI information of (U)SIM card in the corresponding card slot by specifying different (U)SIM IDs, as follows.

```
ql_sim_get_imsi(0, imsi, imsi_len);    //Obtain IMSI information of (U)SIM in slot 1.
ql_sim_get_imsi(1, imsi, imsi_len);    //Obtain IMSI information of (U)SIM in slot 2.
```

- In DSSS, only one (U)SIM card is visible in the application layer. (U)SIM ID is fixed as 0 at API execution. If you need to obtain an IMSI information of another (U)SIM card, you need to manually switch to the desired (U)SIM card slot through `ql_sim_set_operate_id()`, and then read the IMSI information from the (U)SIM card in that slot.

```
ql_sim_get_imsi(0, imsi, imsi_len);    //Obtain IMSI information of (U)SIM in slot 1.
ql_sim_set_operate_id(1);               //Switch (U)SIM card slot.
ql_sim_get_imsi(0, imsi, imsi_len);    //Obtain IMSI information of (U)SIM in slot 2.
```

# 4 (U)SIM API

## 4.1. Header File

*ql\_api\_sim.h*, the header file of (U)SIM card API, is in the *components\ql-kernel\inc* directory. Unless otherwise specified, the header files mentioned in this document are in this directory.

## 4.2. Example File

The example file of (U)SIM card API is in the *components\ql-application\sim\ql\_sim\_demo.c* directory. The example file of phone book API is in the *components\ql-application\sim\ql\_pbk\_demo.c* directory.

## 4.3. API Overview

Table 2: API Overview

| Function                        | Description                                     |
|---------------------------------|-------------------------------------------------|
| <i>ql_sim_get_imsi()</i>        | Gets IMSI of (U)SIM card.                       |
| <i>ql_sim_get_iccid()</i>       | Gets ICCID of (U)SIM card.                      |
| <i>ql_sim_get_phonenumber()</i> | Gets the (U)SIM card phone number.              |
| <i>ql_sim_get_card_status()</i> | Gets the (U)SIM card status.                    |
| <i>ql_sim_enable_pin()</i>      | Enables the PIN verification.                   |
| <i>ql_sim_disable_pin()</i>     | Disables the PIN verification.                  |
| <i>ql_sim_verify_pin()</i>      | Verifies the PIN when (U)SIM card requires PIN. |
| <i>ql_sim_change_pin()</i>      | Changes the PIN.                                |

|                                           |                                                                                          |
|-------------------------------------------|------------------------------------------------------------------------------------------|
| <code>ql_sim_unblock_pin()</code>         | Unlocks the (U)SIM card with PUK and sets a new PIN.                                     |
| <code>ql_sim_set_operate_id()</code>      | Sets the slot ID of (U)SIM card to be used.                                              |
| <code>ql_sim_get_operate_id()</code>      | Gets the slot ID of (U)SIM card currently in use.                                        |
| <code>ql_sim_set_hotplug()</code>         | Configures (U)SIM card hot-plugging detection.                                           |
| <code>ql_sim_hotplug_register_cb()</code> | Registers the callback function for (U)SIM card hot-plugging detection events.           |
| <code>ql_sim_read_pbk_item()</code>       | Reads one or more phone number records from the specified phone book of the (U)SIM card. |
| <code>ql_sim_write_pbk_item()</code>      | Writes phone number records to the specified phone book of the (U)SIM card.              |
| <code>ql_sim_set_pbk_encoding()</code>    | Sets the phone book character set.                                                       |
| <code>ql_sim_get_pbk_encoding()</code>    | Gets the phone book character set.                                                       |
| <code>ql_pbk_callback_register()</code>   | Registers phone book callback function.                                                  |

## 4.4. Description of (U)SIM Card API

### 4.4.1. ql\_sim\_get\_imsi

This function gets IMSI of (U)SIM card.

#### ● Prototype

```
ql_sim_errcode_e ql_sim_get_imsi(uint8_t nSim, char *imsi, size_t imsi_len)
```

#### ● Parameter

*nSim*:

[In] (U)SIM card in use. If the module supports a single (U)SIM card, this parameter can only be set to 0.

0 (U)SIM card 1

1 (U)SIM card 2

*imsi*:

[Out] IMSI of (U)SIM card.

*imsi\_len*:

[In] Length of the *imsi* array.

## Return Value

See **Chapter 4.4.1.1** for details.

### 4.4.1.1. ql\_sim\_errcode\_e

The error codes of (U)SIM card API indicate whether the API is executed successfully or not. If the execution of API fails, then an error code is returned, indicating the cause of the error.

The enumeration of error codes:

```
typedef enum
{
    QL_SIM_SUCCESS                                = 0,
    QL_SIM_EXECUTE_ERR                            = 1|QL_SIM_ERRCODE_BASE,
    QL_SIM_MEM_ADDR_NULL_ERR,
    QL_SIM_INVALID_PARAM_ERR,
    QL_SIM_NO_MEMORY_ERR,
    QL_SIM_SEMAPHORE_CREATE_ERR                    = 5|QL_SIM_ERRCODE_BASE,
    QL_SIM_SEMAPHORE_TIMEOUT_ERR,
    QL_SIM_NOT_INSERTED_ERR,
    QL_SIM_CFW_IMSI_GET_REQUEST_ERR,
    QL_SIM_CFW_IMSI_GET_RSP_ERR,
    QL_SIM_CFW_ICCID_GET_REQUEST_ERR                = 10|QL_SIM_ERRCODE_BASE,
    QL_SIM_CFW_PHONE_NUM_GET_REQUEST_ERR,
    QL_SIM_CFW_PHONE_NUM_GET_RSP_NULL_ERR,
    QL_SIM_CFW_STATUS_GET_REQUEST_ERR,
    QL_SIM_CFW_PIN_ENABLE_REQUEST_ERR,
    QL_SIM_CFW_PIN_DISABLE_REQUEST_ERR              = 15|QL_SIM_ERRCODE_BASE,
    QL_SIM_CFW_PIN_VERIFY_REQUEST_ERR,
    QL_SIM_CFW_PIN_CHANGE_REQUEST_ERR,
    QL_SIM_CFW_PIN_UNBLOCK_REQUEST_ERR,
    QL_SIM_OPERATION_NOT_ALLOWED_ERR,
    QL_PBK_NOT_EXIT_ERR                            = 100|QL_SIM_ERRCODE_BASE,
    QL_PBK_NOT_INIT_ERR,
    QL_PBK_ITEM_NOT_FOUND_ERR,
} ql_sim_errcode_e
```

## Member

| Member         | Description           |
|----------------|-----------------------|
| QL_SIM_SUCCESS | Successful execution. |

|                                       |                                                               |
|---------------------------------------|---------------------------------------------------------------|
| QL_SIM_EXECUTE_ERR                    | Failed execution.                                             |
| QL_SIM_MEM_ADDR_NULL_ERR              | Parameter address is NULL.                                    |
| QL_SIM_INVALID_PARAM_ERR              | Invalid parameter(s).                                         |
| QL_SIM_NO_MEMORY_ERR                  | Failed to apply for the memory.                               |
| QL_SIM_SEMAPHORE_CREATE_ERR           | Failed to create semaphores.                                  |
| QL_SIM_SEMAPHORE_TIMEOUT_ERR          | Semaphore timeout period has expired.                         |
| QL_SIM_NOT_INSERTED_ERR               | (U)SIM card is not inserted.                                  |
| QL_SIM_CFW_IMSI_GET_REQUEST_ERR       | Request for IMSI of the (U)SIM card was rejected.             |
| QL_SIM_CFW_IMSI_GET_RSP_ERR           | Failed to get IMSI of the (U)SIM card.                        |
| QL_SIM_CFW_ICCID_GET_REQUEST_ERR      | Request for ICCID of the (U)SIM card was rejected.            |
| QL_SIM_CFW_PHONE_NUM_GET_REQUEST_ERR  | Request for the phone number of the (U)SIM card was rejected. |
| QL_SIM_CFW_PHONE_NUM_GET_RSP_NULL_ERR | Phone number of (U)SIM card is null.                          |
| QL_SIM_CFW_STATUS_GET_REQUEST_ERR     | Request for the status of the (U)SIM card was rejected.       |
| QL_SIM_CFW_PIN_ENABLE_REQUEST_ERR     | Request to enable (U)SIM card PIN verification failed.        |
| QL_SIM_CFW_PIN_DISABLE_REQUEST_ERR    | Request to disable (U)SIM card PIN verification failed.       |
| QL_SIM_CFW_PIN_VERIFY_REQUEST_ERR     | Request to verify (U)SIM card PIN failed.                     |
| QL_SIM_CFW_PIN_CHANGE_REQUEST_ERR     | Request to change (U)SIM card PIN failed.                     |
| QL_SIM_CFW_PIN_UNBLOCK_REQUEST_ERR    | Request to unlock (U)SIM card with PUK failed.                |
| QL_SIM_OPERATION_NOT_ALLOWED_ERR      | Operation not allowed.                                        |
| QL_PBK_NOT_EXIT_ERR                   | Phone book not existed.                                       |
| QL_PBK_NOT_INIT_ERR                   | Phone book not initialized.                                   |
| QL_PBK_ITEM_NOT_FOUND_ERR             | Phone number record not found.                                |

#### 4.4.2. ql\_sim\_get\_iccid

This function gets ICCID of (U)SIM card.



### ● Prototype

```
ql_sim_errcode_e ql_sim_get_iccid(uint8_t nSim, char *iccid, size_t iccid_len)
```

### ● Parameter

*nSim*:

[In] (U)SIM card in use. If the module supports a single (U)SIM card, this parameter can only be set to 0.

0 (U)SIM card 1

1 (U)SIM card 2

*iccid*:

[Out] ICCID of (U)SIM card.

*iccid\_len*:

[In] Length of the *iccid* array.

### ● Return Value

See **Chapter 4.4.1.1** for details.

## 4.4.3. ql\_sim\_get\_phonenumber

This function gets the (U)SIM card phone number.

### ● Prototype

```
ql_sim_errcode_e ql_sim_get_phonenumber(uint8_t nSim, char *phonenumber, size_t
phonenumber_len)
```

### ● Parameter

*nSim*:

[In] (U)SIM card in use. If the module supports a single (U)SIM card, this parameter can only be set to 0.

0 (U)SIM card 1

1 (U)SIM card 2

*phonenumber*:

[Out] Phone number of (U)SIM card.

*phonenumber\_len*:

[In] Length of the *phonenumber* array.

### ● Return Value

See **Chapter 4.4.1.1** for details.

#### 4.4.4. ql\_sim\_get\_card\_status

This function gets the (U)SIM card status.

##### ● Prototype

```
ql_sim_errcode_e ql_sim_get_card_status(uint8_t nSim, ql_sim_status_e *card_status)
```

##### ● Parameter

*nSim*:

[In] (U)SIM card in use. If the module supports a single (U)SIM card, this parameter can only be set to 0.

0 (U)SIM card 1

1 (U)SIM card 2

*card\_status*:

[Out] (U)SIM card status. See **Chapter 4.4.4.1** for details.

##### ● Return Value

See **Chapter 4.4.1.1** for details.

#### 4.4.4.1. ql\_sim\_status\_e

The enumeration of (U)SIM card status:

```
typedef enum
{
    QL_SIM_STATUS_READY                = 0,
    QL_SIM_STATUS_PIN1_READY,
    QL_SIM_STATUS_SIMPIN,
    QL_SIM_STATUS_SIMPUK,
    QL_SIM_STATUS_PHONE_TO_SIMPIN,
    QL_SIM_STATUS_PHONE_TO_FIRST_SIMPIN = 5,
    QL_SIM_STATUS_PHONE_TO_FIRST_SIMPUK,
    QL_SIM_STATUS_SIMPIN2,
    QL_SIM_STATUS_SIMPUK2,
    QL_SIM_STATUS_NETWORKPIN,
    QL_SIM_STATUS_NETWORKPUK           = 10,
    QL_SIM_STATUS_NETWORK_SUBSETPIN,
    QL_SIM_STATUS_NETWORK_SUBSETPUK,
    QL_SIM_STATUS_PROVIDERPIN,
    QL_SIM_STATUS_PROVIDERPUK,
    QL_SIM_STATUS_CORPORATEPIN         = 15,
```

```

QL_SIM_STATUS_CORPORATEPUK,
QL_SIM_STATUS_NOSIM,
QL_SIM_STATUS_PIN1BLOCK,
QL_SIM_STATUS_PIN2BLOCK,
QL_SIM_STATUS_PIN1_DISABLE    = 20,
QL_SIM_STATUS_SIM_PRESENT,
QL_SIM_STATUS_UNKNOW,
}ql_sim_status_e

```

### ● Member

| Member                                     | Description                                                                                                                                                       |
|--------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>QL_SIM_STATUS_READY</i>                 | (U)SIM card is ready.                                                                                                                                             |
| <i>QL_SIM_STATUS_PIN1_READY</i>            | PIN1 is ready.                                                                                                                                                    |
| <i>QL_SIM_STATUS_SIMPIN</i>                | (U)SIM card is locked and PIN is required.                                                                                                                        |
| <i>QL_SIM_STATUS_SIMPUK</i>                | (U)SIM card is blocked and PUK is required.                                                                                                                       |
| <i>QL_SIM_STATUS_PHONE_TO_SIMPIN</i>       | (U)SIM card is locked due to (U)SIM card personalization check failure. (U)SIM card is waiting for a PCK to deactivate (U)SIM card personalization.               |
| <i>QL_SIM_STATUS_PHONE_TO_FIRST_SIMPIN</i> | Expecting key for hidden phone book entries.                                                                                                                      |
| <i>QL_SIM_STATUS_PHONE_TO_FIRST_SIMPUK</i> | Expecting code to unblock the hidden key.                                                                                                                         |
| <i>QL_SIM_STATUS_SIMPIN2</i>               | (U)SIM card is locked and PIN2 is required.                                                                                                                       |
| <i>QL_SIM_STATUS_SIMPUK2</i>               | (U)SIM card is blocked and PUK2 is required.                                                                                                                      |
| <i>QL_SIM_STATUS_NETWORKPIN</i>            | (U)SIM card is locked due to network personalization check failure. (U)SIM card is waiting for an NCK to deactivate the network personalization.                  |
| <i>QL_SIM_STATUS_NETWORKPUK</i>            | (U)SIM card is blocked due to an incorrect NCK. An MEP unblocking password is required.                                                                           |
| <i>QL_SIM_STATUS_NETWORK_SUBSETPIN</i>     | (U)SIM card is locked due to a network subset personalization check failure. (U)SIM card is waiting for an NSCK to deactivate the network subset personalization. |
| <i>QL_SIM_STATUS_NETWORK_SUBSETPUK</i>     | (U)SIM card is blocked due to an incorrect NSCK. An MEP unblocking password is required.                                                                          |
| <i>QL_SIM_STATUS_PROVIDERPIN</i>           | (U)SIM card is locked due to a service provider personalization check failure. (U)SIM card is waiting for an SPCK to deactivate the service provider              |

|                                         |                                                                                                                                                       |
|-----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                         | personalization.                                                                                                                                      |
| <code>QL_SIM_STATUS_PROVIDERPUK</code>  | (U)SIM card is blocked due to an incorrect SPCK. An MEP unblocking password is required.                                                              |
| <code>QL_SIM_STATUS_CORPORATEPIN</code> | (U)SIM card is locked due to a corporate personalization check failure. (U)SIM card is waiting for a CCK to deactivate the corporate personalization. |
| <code>QL_SIM_STATUS_CORPORATEPUK</code> | (U)SIM card is blocked due to an incorrect CCK. An MEP unblocking password is required.                                                               |
| <code>QL_SIM_STATUS_NOSIM</code>        | (U)SIM card is not inserted.                                                                                                                          |
| <code>QL_SIM_STATUS_PIN1BLOCK</code>    | Not supported.                                                                                                                                        |
| <code>QL_SIM_STATUS_PIN2BLOCK</code>    | Not supported.                                                                                                                                        |
| <code>QL_SIM_STATUS_PIN1_DISABLE</code> | Not supported.                                                                                                                                        |
| <code>QL_SIM_STATUS_SIM_PRESENT</code>  | Not supported.                                                                                                                                        |
| <code>QL_SIM_STATUS_UNKNOWN</code>      | Unknown.                                                                                                                                              |

#### 4.4.5. ql\_sim\_enable\_pin

This function enables the PIN verification.

##### ● Prototype

```
ql_sim_errcode_e ql_sim_enable_pin(uint8_t nSim, ql_sim_verify_pin_info_s *pt_info)
```

##### ● Parameter

*nSim*:

[In] (U)SIM card in use. If the module supports a single (U)SIM card, this parameter can only be set to 0.

0 (U)SIM card 1

1 (U)SIM card 2

*pt\_info*:

[In] PIN. See **Chapter 4.4.5.1** for details.

##### ● Return Value

See **Chapter 4.4.1.1** for details.

#### 4.4.5.1. ql\_sim\_verify\_pin\_info\_s

The structure of the PIN:

```
typedef struct
{
    uint8_t pin_value[QL_SIM_PIN_LEN_MAX+1];
}ql_sim_verify_pin_info_s
```

##### ● Parameter

| Type    | Parameter        | Description                                                            |
|---------|------------------|------------------------------------------------------------------------|
| uint8_t | <i>pin_value</i> | String type. PIN. Maximum length: <i>QL_SIM_PIN_LEN_MAX</i> (8) bytes. |

#### 4.4.6. ql\_sim\_disable\_pin

This function disables the PIN verification.

##### ● Prototype

```
ql_sim_errcode_e ql_sim_disable_pin(uint8_t nSim, ql_sim_verify_pin_info_s *pt_info)
```

##### ● Parameter

*nSim*:

[In] (U)SIM card in use. If the module supports a single (U)SIM card, this parameter can only be set to 0.

0 (U)SIM card 1

1 (U)SIM card 2

*pt\_info*:

[In] PIN. See **Chapter 4.4.5.1** for details.

##### ● Return Value

See **Chapter 4.4.1.1** for details.

#### 4.4.7. ql\_sim\_verify\_pin

This function verifies the PIN when (U)SIM card requires PIN.

##### ● Prototype

```
ql_sim_errcode_e ql_sim_verify_pin(uint8_t nSim, ql_sim_verify_pin_info_s *pt_info)
```

- **Parameter**

*nSim*:

[In] (U)SIM card in use. If the module supports a single (U)SIM card, this parameter can only be set to 0.

- 0 (U)SIM card 1
- 1 (U)SIM card 2

*pt\_info*:

[In] PIN. See **Chapter 4.4.5.1** for details.

- **Return Value**

See **Chapter 4.4.1.1** for details.

#### 4.4.8. ql\_sim\_change\_pin

This function changes the PIN.

- **Prototype**

```
ql_sim_errcode_e ql_sim_change_pin(uint8_t nSim, ql_sim_change_pin_info_s *pt_info)
```

- **Parameter**

*nSim*:

[In] (U)SIM card in use. If the module supports a single (U)SIM card, this parameter can only be set to 0.

- 0 (U)SIM card 1
- 1 (U)SIM card 2

*pt\_info*:

[In] Old and new PIN. See **Chapter 4.4.8.1** for details.

- **Return Value**

See **Chapter 4.4.1.1** for details.

##### 4.4.8.1. ql\_sim\_change\_pin\_info\_s

The structure of old and new PIN:

```
typedef struct
{
    uint8_t    old_pin_value[QL_SIM_PIN_LEN_MAX+1];
    uint8_t    new_pin_value[QL_SIM_PIN_LEN_MAX+1];
}ql_sim_change_pin_info_s
```

- Parameter

| Type    | Parameter            | Description                                                                    |
|---------|----------------------|--------------------------------------------------------------------------------|
| uint8_t | <i>old_pin_value</i> | String type. Old PIN. Maximum length: <i>QL_SIM_PIN_LEN_MAX</i> (8) bytes.     |
| uint8_t | <i>new_pin_value</i> | String type. The new PIN. Maximum length: <i>QL_SIM_PIN_LEN_MAX</i> (8) bytes. |

#### 4.4.9. ql\_sim\_unblock\_pin

This function unlocks the (U)SIM card with PUK and sets a new PIN. If you enter the incorrect PIN multiple consecutive times, the PIN will be locked, and you will need the PUK to unblock the PIN.

- Prototype

```
ql_sim_errcode_e ql_sim_unblock_pin(uint8_t nSim, ql_sim_unblock_pin_info_s *pt_info)
```

- Parameter

*nSim*:

[In] (U)SIM card in use. If the module supports a single (U)SIM card, this parameter can only be set to 0.

0 (U)SIM card 1

1 (U)SIM card 2

*pt\_info*:

[In] PUK and new PIN. See **Chapter 4.4.9.1** for details.

- Return Value

See **Chapter 4.4.1.1** for details.

##### 4.4.9.1. ql\_sim\_unblock\_pin\_info\_s

The structure of PUK and new PIN:

```
typedef struct
{
    uint8_t puk_value[QL_SIM_PIN_LEN_MAX+1];
    uint8_t new_pin_value[QL_SIM_PIN_LEN_MAX+1];
}ql_sim_unblock_pin_info_s
```

- **Parameter**

| Type    | Parameter            | Description                                                                |
|---------|----------------------|----------------------------------------------------------------------------|
| uint8_t | <i>puk_value</i>     | String type. PUK. Maximum length: <i>QL_SIM_PIN_LEN_MAX</i> (8) bytes.     |
| uint8_t | <i>new_pin_value</i> | String type. New PIN. Maximum length: <i>QL_SIM_PIN_LEN_MAX</i> (8) bytes. |

#### 4.4.10. ql\_sim\_set\_operate\_id

This function sets the slot ID of (U)SIM card to be used. The default value is 0. This function is specifically designed for DSDS applications, allowing you to switch between the slots of (U)SIM cards to be used.

- **Prototype**

```
ql_sim_errcode_e ql_sim_set_operate_id(uint8_t nSim)
```

- **Parameter**

*nSim*:

[In] (U)SIM card slot ID, corresponding to slot in **Figure 2**.

- 0 (U)SIM card slot 1
- 1 (U)SIM card slot 2

- **Return Value**

See **Chapter 4.4.1.1**.

#### 4.4.11. ql\_sim\_get\_operate\_id

This function gets the slot ID of (U)SIM card currently in use and is only intended for DSDS applications.

- **Prototype**

```
uint8_t ql_sim_get_operate_id(void)
```

- **Parameter**

None

- **Return Value**

- 0 (U)SIM card slot 1
- 1 (U)SIM card slot 2



#### 4.4.12. ql\_sim\_set\_hotplug

This function configures (U)SIM card hot-plugging detection.

##### ● Prototype

```
ql_sim_errcode_e ql_sim_set_hotplug(uint8_t nSim, ql_sim_hotplug_e status, ql_sim_hotplug_gpio_s*
gpio_cfg)
```

##### ● Parameter

*nSim*:

[In] (U)SIM card slot ID, corresponding to slot in **Figure 2**.

- 0 (U)SIM card slot 1
- 1 (U)SIM card slot 2

*status*:

[In] Hot-plugging detection status. See **Chapter 4.4.12.1** for details.

*gpio\_cfg*:

[In] Configuration of hot-plugging detection pin. See **Chapter 4.4.12.2** for details.

##### ● Return Value

See **Chapter 4.4.1.1** for details.

#### 4.4.12.1. ql\_sim\_hotplug\_e

The enumeration of hot-plugging detection status:

```
typedef enum
{
    QL_SIM_HOTPLUG_DISABLE          = 0,
    QL_SIM_HOTPLUG_ENABLE           = 1,
}ql_sim_hotplug_e
```

##### ● Member

| Member                        | Description                     |
|-------------------------------|---------------------------------|
| <i>QL_SIM_HOTPLUG_DISABLE</i> | Disable hot-plugging detection. |
| <i>QL_SIM_HOTPLUG_ENABLE</i>  | Enable hot-plugging detection.  |

#### 4.4.12.2. ql\_sim\_hotplug\_gpio\_s

The structure of hot-plugging detection pin configuration:

```
typedef struct
{
    ql_GpioNum gpio_num;
    ql_LvlMode gpio_level;
}ql_sim_hotplug_gpio_s
```

- **Parameter**

| Type              | Parameter         | Description                                                               |
|-------------------|-------------------|---------------------------------------------------------------------------|
| <i>ql_GpioNum</i> | <i>gpio_num</i>   | GPIO pin number. See <b>document [2]</b> for details.                     |
| <i>ql_LvlMode</i> | <i>gpio_level</i> | Pin level, available for output pin. See <b>document [2]</b> for details. |

#### 4.4.13. ql\_sim\_hotplug\_register\_cb

This function registers the callback function for (U)SIM card hot-plugging detection events.

- **Prototype**

```
ql_sim_errcode_e ql_sim_hotplug_register_cb(ql_sim_hotplug_cb_t cb)
```

- **Parameter**

*cb*:

[In] Callback function for (U)SIM card hot-plugging detection events. See **Chapter 4.4.13.1** for details.

- **Return Value**

See **Chapter 4.4.1.1** for details.

##### 4.4.13.1. ql\_sim\_hotplug\_cb\_t

This is a callback function for (U)SIM card hot-plugging detection events.

- **Prototype**

```
typedef void (*ql_sim_hotplug_cb_t)(uint8_t nSim, ql_sim_hotplug_status_e status)
```

- **Parameter**

*nSim*:

[In] (U)SIM card slot ID, corresponding to slot in **Figure 2**.

- 0 (U)SIM card slot 1
- 1 (U)SIM card slot 2

*status*:

[In] (U)SIM card hot-plugging pin status. See **Chapter 4.4.13.2** for details.

- **Return Value**

None

#### 4.4.13.2. ql\_sim\_hotplug\_status\_e

The enumeration of (U)SIM card hot-plugging status:

```
typedef enum
{
    QL_SIM_HOTPLUG_STATUS_OUT    = 0,
    QL_SIM_HOTPLUG_STATUS_IN    = 1,
}ql_sim_errcode_e
```

- **Member**

| Member                    | Description              |
|---------------------------|--------------------------|
| QL_SIM_HOTPLUG_STATUS_OUT | (U)SIM card is removed.  |
| QL_SIM_HOTPLUG_STATUS_IN  | (U)SIM card is inserted. |

#### 4.4.14. ql\_sim\_read\_pbk\_item

This function reads one or more phone number records from the specified phone book of the (U)SIM card.

- **Prototype**

```
ql_sim_errcode_e ql_sim_read_pbk_item(uint8_t nSim,
                                       ql_sim_pbk_storage_e storage,
                                       int start_index,
                                       int end_index,
                                       uint8_t *username,
                                       uint8_t username_len,
                                       ql_sim_pbk_itemset_info_s *itemset)
```

## ● Parameter

*nSim*:

[In] (U)SIM card in use. If the module supports a single (U)SIM card, this parameter can only be set to 0.

0 (U)SIM card 1

1 (U)SIM card 2

*storage*:

[In] Storage location of the phone number in the phone book. See **Chapter 4.4.14.1** for details.

*start\_index*:

[In] Starting number of the phone number record to be read. 0 means that the number is not used to get the phone number record.

*end\_index*:

[In] Ending number of the phone number record to be read.

*username*:

[In] Username in the phone number record. It is valid when *start\_index* is 0.

*username\_len*:

[In] Length of the username in the phone number record. Unit: characters. The parameter is valid when *start\_index* is 0.

*itemset*:

[Out] List of phone number records to be read. See **Chapter 4.4.14.2** for details.

## ● Return Value

See **Chapter 4.4.1.1** for details.

### 4.4.14.1. ql\_sim\_pbk\_storage\_e

The enumeration of storage locations of the phone number in the phone book:

```
typedef enum
{
    QL_SIM_PBK_STORAGE_FD,
    QL_SIM_PBK_STORAGE_LD,
    QL_SIM_PBK_STORAGE_ME,
    QL_SIM_PBK_STORAGE_ON,
    QL_SIM_PBK_STORAGE_SM,
    QL_SIM_PBK_STORAGE_MAX,
} ql_sim_pbk_storage_e
```

- Member

| Member                             | Description                                        |
|------------------------------------|----------------------------------------------------|
| <code>QL_SIM_PBK_STORAGE_FD</code> | Fixed dialing phone book of (U)SIM card            |
| <code>QL_SIM_PBK_STORAGE_LD</code> | The latest dialing phone book of (U)SIM card       |
| <code>QL_SIM_PBK_STORAGE_ME</code> | MT phone book                                      |
| <code>QL_SIM_PBK_STORAGE_ON</code> | Phone number (MSISDNs) list of (U)SIM card (or MT) |
| <code>QL_SIM_PBK_STORAGE_SM</code> | SIM/UICC phone book                                |

#### 4.4.14.2. `ql_sim_pbk_itemset_info_s`

The structure of the phone number record list to be read:

```
typedef struct
{
    int item_count;
    ql_sim_pbk_item_info_s item[QL_PBK_ITEM_COUNT_MAX];
} ql_sim_pbk_itemset_info_s
```

- Parameter

| Type                          | Parameter         | Description                                                                  |
|-------------------------------|-------------------|------------------------------------------------------------------------------|
| int                           | <i>item_count</i> | Phone number record count.                                                   |
| <i>ql_sim_pbk_item_info_s</i> | <i>item</i>       | Details of phone number records.<br>See <b>Chapter 4.4.15.1</b> for details. |

#### 4.4.15. `ql_sim_write_pbk_item`

This function writes phone number records to the specified phone book of the (U)SIM card. This function can write phone number records multiple times, but only one record at a time.

- Prototype

```
ql_sim_errcode_e ql_sim_write_pbk_item(uint8_t nSim, ql_sim_pbk_storage_e storage, ql_sim_pbk_item_info_s *item)
```

### ● Parameter

*nSim*:

[In] (U)SIM card in use. If the module supports a single (U)SIM card, this parameter can only be set to 0.

0 (U)SIM card 1

1 (U)SIM card 2

*storage*:

[In] Storage location of the phone number in the phone book. See **Chapter 4.4.14.1** for details.

*item*:

[In] Phone number records to be written. See **Chapter 4.4.15.1** for details.

### ● Return Value

See **Chapter 4.4.1.1** for details.

#### 4.4.15.1. ql\_sim\_pbk\_item\_info\_s

The structure of the phone number records to be written:

```
typedef struct
{
    int index;
    uint8_t username[QL_PBK_USERNAME_LEN_MAX];
    uint8_t phonenumber[QL_PBK_PHONENUM_LEN_MAX];
    uint8_t username_len;
} ql_sim_pbk_item_info_s
```

### ● Parameter

| Type    | Parameter          | Description                                                                                                                                                                                                                                                                         |
|---------|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| int     | <i>index</i>       | The number of the phone number records in the phone book. When you write a phone number record, if <i>index</i> is 0, it means that the phone number record count is automatically set. If <i>index</i> is not 0, it means that the record is written with the <i>index</i> number. |
| uint8_t | <i>username</i>    | The username in the phone number record. Default and maximum length: 32 characters. It uses the encoding methods listed in <i>ql_sim_pbk_encoding_e</i> , see <b>Chapter 4.4.16.1</b> for details.                                                                                  |
| uint8_t | <i>phonenumber</i> | The phone number in the phone number record, a string which may contain "+". Default and maximum length: 24 characters. When <i>phonenumber</i> is null, the phone number record specified by <i>index</i> will be deleted.                                                         |

|         |              |                                                                      |
|---------|--------------|----------------------------------------------------------------------|
| uint8_t | username_len | Length of the username in the phone number record. Unit: characters. |
|---------|--------------|----------------------------------------------------------------------|

#### 4.4.16. ql\_sim\_set\_pbk\_encoding

This function sets the phone number character set. The phone number character set will be recovered to *QL\_SIM\_PBK\_GSM* after the module is rebooted.

##### ● Prototype

```
ql_sim_errcode_e ql_sim_set_pbk_encoding(ql_sim_pbk_encoding_e encoding)
```

##### ● Parameter

*encoding*:

[In] Phone book character set. See **Chapter 4.4.16.1** for details.

##### ● Return Value

See **Chapter 4.4.1.1** for details.

#### 4.4.16.1. ql\_sim\_pbk\_encoding\_e

The enumeration of the phone book character sets:

```
typedef enum
{
    QL_SIM_PBK_GSM,
    QL_SIM_PBK_HEX,
    QL_SIM_PBK_UCS2,
    QL_SIM_PBK_GBK,
    QL_SIM_PBK_IRA,
    QL_SIM_PBK_ENCODE_MAX,
} ql_sim_pbk_encoding_e
```

##### ● Member

| Member                 | Description                                                          |
|------------------------|----------------------------------------------------------------------|
| <i>QL_SIM_PBK_GSM</i>  | GSM character set (default)                                          |
| <i>QL_SIM_PBK_HEX</i>  | Character strings only composed of hexadecimal strings from 00 to FF |
| <i>QL_SIM_PBK_UCS2</i> | UCS2 character set                                                   |

|                             |     |
|-----------------------------|-----|
| <code>QL_SIM_PBK_GBK</code> | GBK |
| <code>QL_SIM_PBK_IRA</code> | IRA |

#### 4.4.17. ql\_sim\_get\_pbk\_encoding

This function gets the phone book character set. The default phone book character set is `QL_SIM_PBK_GSM`.

- **Prototype**

```
ql_sim_errcode_e ql_sim_get_pbk_encoding(ql_sim_pbk_encoding_e *encoding)
```

- **Parameter**

*encoding*:

[In] Phone book character set. See **Chapter 4.4.16.1** for details.

- **Return Value**

See **Chapter 4.4.1.1** for details.

#### 4.4.18. ql\_pbk\_callback\_register

This function registers the phone book callback function.

- **Prototype**

```
void ql_pbk_callback_register(ql_pbk_event_handler_t cb)
```

- **Parameter**

*cb*:

[In] Callback function for phone book-related events. See **Chapter 4.4.18.1** for details.

- **Return Value**

None

##### 4.4.18.1. ql\_pbk\_event\_handler\_t

This function is a callback function for phone book-related events.



- **Prototype**

```
typedef void (*ql_pbk_event_handler_t)(uint8_t nSim, int event_id, void *ctx)
```

- **Parameter**

*nSim*:

[In] (U)SIM card in use. If the module supports a single (U)SIM card, this parameter can only be set to 0.

0 (U)SIM card 1

1 (U)SIM card 2

*event\_id*:

[In] Phone book event ID.

QL\_PBK\_INIT\_OK\_IND Phone book is initialized successfully.

*ctx*:

[In] Phone book event context.

- **Return Value**

None

# 5 Appendix References

**Table 3: Related Documents**

| Document Name                                                         |
|-----------------------------------------------------------------------|
| [1] Quectel_EC200U&EG91xU_Series_QuecOpen(SDK)_Quick_Start_Guide      |
| [2] Quectel_EC200U&EG91xU_Series_QuecOpen(SDK)_GPIO_Development_Guide |

**Table 4: Terms and Abbreviations**

| Abbreviation | Description                              |
|--------------|------------------------------------------|
| API          | Application Programming Interface        |
| CCK          | Corporate Control Key                    |
| GBK          | Chinese Internal Code Specification      |
| GSM          | Global System for Mobile Communications  |
| HEX          | Hexadecimal                              |
| ICCID        | Integrate Circuit Card Identity          |
| IMSI         | International Mobile Subscriber Identity |
| IoT          | Internet of Things                       |
| IRA          | International Reference Alphabet         |
| MEP          | Mobile Equipment Personalization         |
| MT           | Mobile Termination                       |
| NSCK         | Network Subset Control Key               |
| NSISDN       | Mobile Subscriber ISDN Number            |
| PCK          | Personalization Control Key              |
| PIN          | Personal Identification Number           |

---

|        |                                           |
|--------|-------------------------------------------|
| PUK    | Personal Identification Number Unlock Key |
| RTOS   | Real-Time Operating System                |
| SDK    | Software Development Kit                  |
| SIM    | Subscriber Identity Module                |
| UCS    | Universal Character Set                   |
| UICC   | Universal Integrated Circuit Card         |
| (U)SIM | (Universal) Subscriber Identity Module    |

---